GROUP NAME: CODE MONKEYS **SUBJECT:** IT 105 Information Management I

COURSE, YEAR & SECTION: BSIS 2A **PROFESSOR:** Dr. GUILLERMO V. RED, Jr.

Reflection on Emerging Database Technologies

When we first built our Student Management System, we honestly thought, "It works, so why change it?" But after researching emerging database technologies for this phase, we realized that "working" and "working well for the future" are two different things. Two trends caught our attention: **serverless SQL databases** and **in-database machine learning** (often called ML-integrated databases). They could make our project smarter and more resilient, though they also come with real trade-offs.

Serverless SQL databases (like Amazon Aurora Serverless or PlanetScale) automatically scale compute and storage based on actual usage. For our enrollment system, that's a game changer. Right now, if 200 students try to enroll at the same time, our MySQL instance might slow down. With serverless, the database would instantly add resources to handle the spike and then scale back down during quiet hours to save cost (Amazon Web Services, n.d.; PlanetScale, n.d.). More importantly, serverless platforms often include built-in backup and point-in-time recovery. That would automate the mysqldump backups we did manually in Phase 4—fewer things to forget during a real disaster.

ML-integrated databases (e.g., BigQuery ML) let us train and run machine learning models directly inside the database using SQL (Google Cloud, n.d.). Applied to our project, we could predict which courses will be overbooked next semester or identify students at risk of failing based on grade patterns. Guo et al. (2025) show that embedding ML predicates into SQL queries can make these predictions almost real-time. Instead of exporting data to Python scripts, we would keep everything inside the database simpler architecture, less data movement, and faster insights. Imagine our BI dashboard automatically flagging a warning: "Course X has a 90% chance of exceeding capacity next month." That's a real upgrade from static charts.

Serverless isn't magic. One well-known downside is **cold start latency** if no one uses the system for an hour, the first query after that can take several seconds to respond (Swenson et al., 2025). In a school registrar's office, that would be frustrating. Also, some serverless databases restrict certain SQL features (like LOCK TABLES) and, if misconfigured, costs can spike unpredictably during a busy enrollment week.

For in-database ML, the biggest barrier is **data quality and volume**. Our 100,000+ records might be enough for basic predictions, but training a reliable model also requires clean, consistent data and meaningful features (e.g., attendance, assignment scores) that we don't track yet. Guo et al. (2025) note that ML predicates can actually slow down queries if the underlying data isn't well-structured. And once we start using a cloud vendor's ML tools, migrating away later becomes difficult that's vendor lock-in.

For our group project right now, sticking with traditional MySQL is completely fine. But learning about serverless and ML-integrated databases changed how we think about "done." If we were building this for a real school, we would seriously consider a hybrid approach: keep core transactional data on standard MySQL, but replicate enrollment and grade data to a serverless, ML-ready cloud database for analytics and predictions. It's not a simple upgrade there are costs and learning curves but the potential to move from *reporting what happened* to *predicting what will happen* is worth the effort.

References:

Amazon Web Services. (n.d.). *Amazon Aurora Serverless*. Retrieved April 17, 2026, from https://aws.amazon.com/rds/aurora/serverless/

Google Cloud. (n.d.). *BigQuery ML documentation*. Retrieved April 17, 2026, from https://cloud.google.com/bigquery/docs/bigqueryml-intro

Guo, Y., Li, G., Hu, R., & Wang, Y. (2025). In-database query optimization on SQL with ML predicates. *The VLDB Journal*, *34*(1), Article 12. https://doi.org/10.1007/s00778-024-00888-3

PlanetScale. (n.d.). *PlanetScale documentation*. Retrieved April 17, 2026, from https://planetscale.com/docs

Swenson, J., Kimball, A., Sharif, I., Taft, R., et al. (2025). CockroachDB Serverless: Sub-second scaling from zero with multi-region cluster virtualization. In *Companion of the 2025 International Conference on Management of Data (SIGMOD/PODS 2025)* (pp. 648-661). ACM. https://doi.org/10.1145/3723281.3724163